

TECHNICAL ASSESSMENT REPORT

Full Stack Developer Assessment — ZappySales Project

Candidate Name: Ngane Emmanuel
Role: Full Stack Developer Assessment
Project Name: ZappySales User & Address Management
Date of Submission: June 17, 2026

1. Executive Summary

ZappySales is a production-quality, lightweight full-stack administrative application designed to allow system administrators to view, search, paginate, and modify user registration profiles and their associated shipping/billing addresses.

The application follows the **Aggregate Root** pattern: the User is the parent record, and multiple Address items are managed directly within the User's context (a 1-to-many relationship).

This assessment implementation goes beyond a minimal proof-of-concept by introducing:

- **Server-side Pagination and Case-insensitive Substring Search** (supporting enterprise-scale data sets).
- **Environment-driven properties** with standard fallback defaults for cloud-native deployment.
- **IP-based in-memory Rate Limiting / Request Throttling** on the backend.
- **35 Vitest + React Testing Library (RTL) tests** on the frontend.
- **47 JUnit 5 + Mockito + MockMvc unit/integration tests** on the backend.

2. Technical Stack & Architecture

Backend (Java / Spring Boot)

The backend is structured around a clean, layered architectural design (**Separation of Concerns**):

```
UserController → UserService / UserServiceImpl → UserRepository /  
InMemoryUserRepository
```

- **Language/Framework:** Java 21, Spring Boot v4.1.0.
- **Thread-safe Runtime Store:** Managed in-memory via ConcurrentHashMap with defensive cloning during reads/writes to isolate repository state.
- **Data Transfer Objects (DTOs):** Strong contracts (CreateUserRequest, UserResponse, AddressResponse, etc.) separate the external API model from internal domain models.
- **API Validation:** Jakarta Bean constraints (@Email, @Size, @NotBlank) integrated with custom HTML input sanitizers (@SanitizedString).
- **Security & Throttling:** Custom servlet filter applying security headers (X-Content-Type-Options, X-Frame-Options, Referrer-Policy) and IP-based sliding-window request rate limiting.

Frontend (React / TypeScript)

- **Framework & Tooling:** React 19, Vite 8, TypeScript 6.
- **Styling System:** TailwindCSS v4.0 for utility layout grids and layout properties combined with Material-UI (MUI) v9 for UI component blocks.
- **State Management:** Zustand state store (userStore.ts) providing reactive global flow with minimal boilerplate.
- **Data Fetching:** Standardized Axios client wrapper (client.ts) with custom interceptors to map error codes and normalize response structures.

3. Directory Layout

```
ZappySales/
├── backend/                                # Spring Boot Project
│   ├── src/main/java/                     # Core Source Code
│   ├── src/main/resources/               # application.properties
│   ├── src/test/java/                   # JUnit Test Suites
│   └── frontend/                         # React + TypeScript Client
│       ├── src/
│       │   ├── api/                     # Centralized Axios Client
│       │   ├── app/routes/              # URL Routing Layouts
│       │   ├── features/users/          # Pages, Components, Stores, and Hooks
│       │   └── shared/                  # Common Page Skeletons, Layouts, and Spinners
│       ├── test/                         # Vitest Setup Configs
│       ├── vitest.config.ts              # Vitest Config File
│       └── README.md                     # Setup and Running Guide
```

4. Environment & Prerequisites

- **Java Development Kit (JDK) 21**
- **Node.js (v18 or higher)** and **npm**
- **Apache Maven** (or use the packaged `./mvnw` wrapper)

Environment Variable	Description	Default Value
<code>PORT</code>	The HTTP port the backend server listens on	<code>8080</code>
<code>ALLOWED_ORIGINS</code>	Comma-separated list of permitted CORS origins	<code>http://localhost:3000,http://localhost:5173</code>
<code>RATE_LIMIT_CAPACITY</code>	Max API requests allowed per client IP within the window	<code>100</code>
<code>RATE_LIMIT_TIME_WINDOW_SECONDS</code>	Throttling time window in seconds	<code>60</code>

Frontend .env Config: Create `frontend/.env` with: `VITE_API_BASE_URL=http://localhost:8080/api/v1`

5. Build, Test, and Execution Instructions

Backend Commands (in `backend/` folder):

```
Run Tests: mvn test
Start Application: ./mvnw spring-boot:run
```

Frontend Commands (in `frontend/` folder):

```
Install Dependencies: npm install
Run Tests: npm run test
Production Build: npm run build
Start Dev Server: npm run dev
```

6. Design Choices: "User → Address" Flow

- **Aggregate Root UI:** Administrators load user list page with server pagination. Clicking details loads the User profile detail page.
- **In-Context Modals:** CRUD tasks (edit profile, CRUD addresses) trigger popup dialogs in-context on the detail view rather than redirecting away, yielding a smooth single-dashboard flow.
- **Zustand Sync:** Successful changes trigger local store mutations rather than forcing database refetches, reducing network utilization.

7. Test Suite Validation Results

- **Backend (47 tests passed):** Checked repository deep cloning, controller endpoints, validation, and full user integration workflows.
- **Frontend (35 tests passed):** Checked table renders, dialog state triggers, Zustand stores, and routing page displays.

Candidate Signature: Ngane Emmanuel • Full Stack Developer